UC San Diego

DSC 102
Systems for Scalable Analytics

Alexander Kolovos

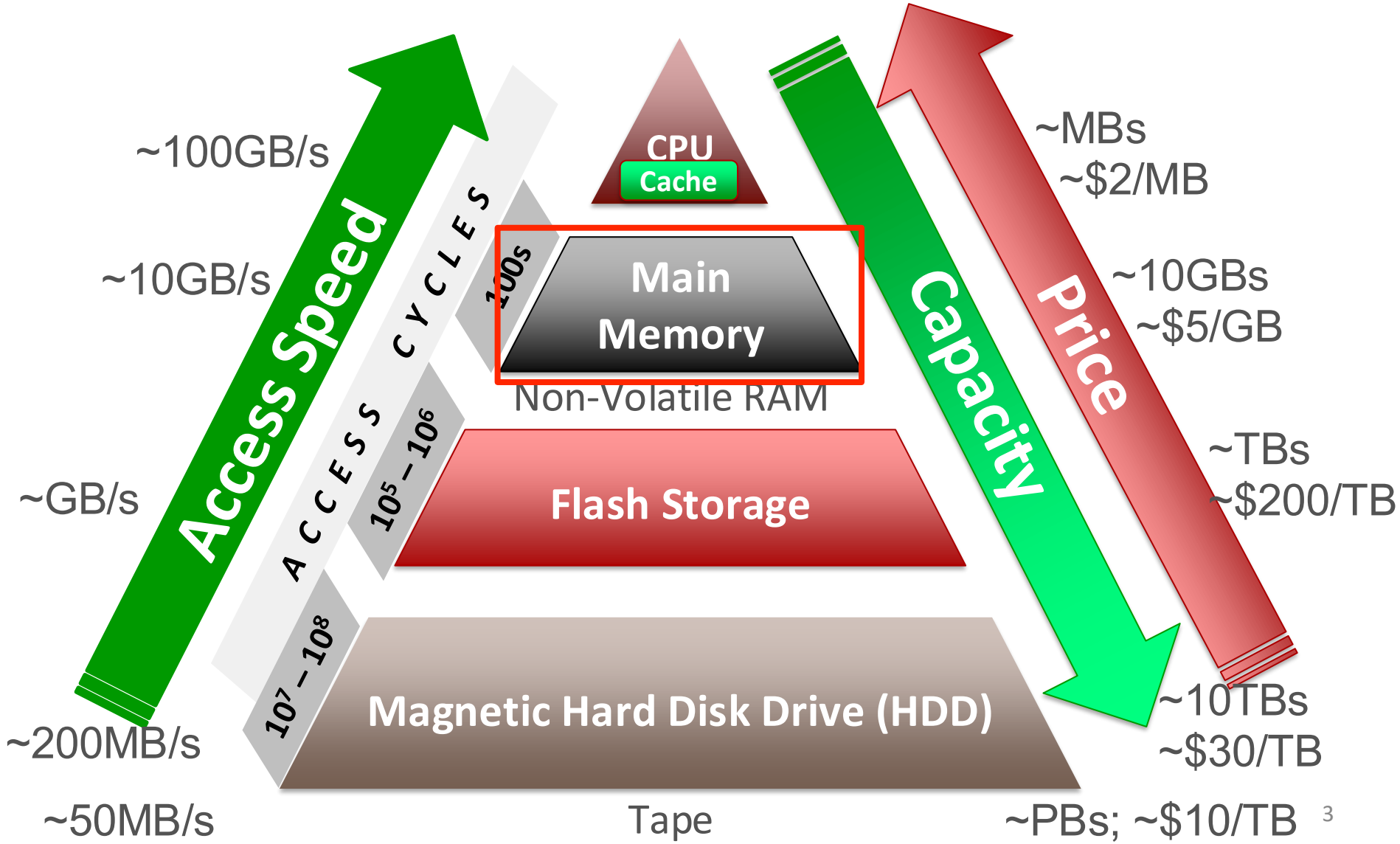
Topic 1: Basics of Machine Resources
Part 2: Operating Systems

Ch. 2, 4.1-4.2, 6, 7, 13, 14.1, 18.1, 21, 22, 26, 36, 37, 39, and 40.1-40.2 of Comet Book

Topic 1 – Outline

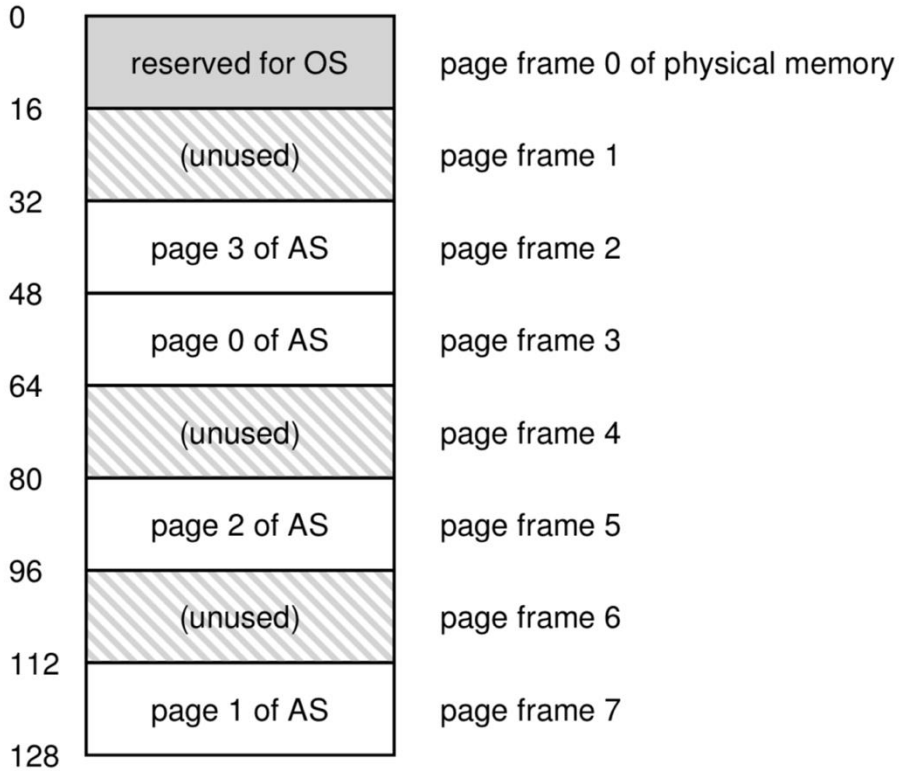
- Basics of Computer Organization
  - Digital Representation of Data
  - Processors and Memory Hierarchy
- Basics of Operating Systems (OS)
  - Process Management: Virtualization; Concurrency
  - Main Memory Management
  - Filesystem and Data Files
- Persistent Data Storage

Memory/Storage Hierarchy



Virtualization of DRAM with Pages

- **Page**: An abstraction of *fixed* size chunks of memory/storage
  - Makes it easier to virtualize and manage DRAM
  - Usually OS configuration parameter; e.g., 4 KB – 16 KB
- **Page Frame**: Virtual slot in DRAM to hold a page's content
- OS **Memory Management** is a system with mechanisms to:
  - Identify pages uniquely
  - Read/write page from/to disk when requested by a process



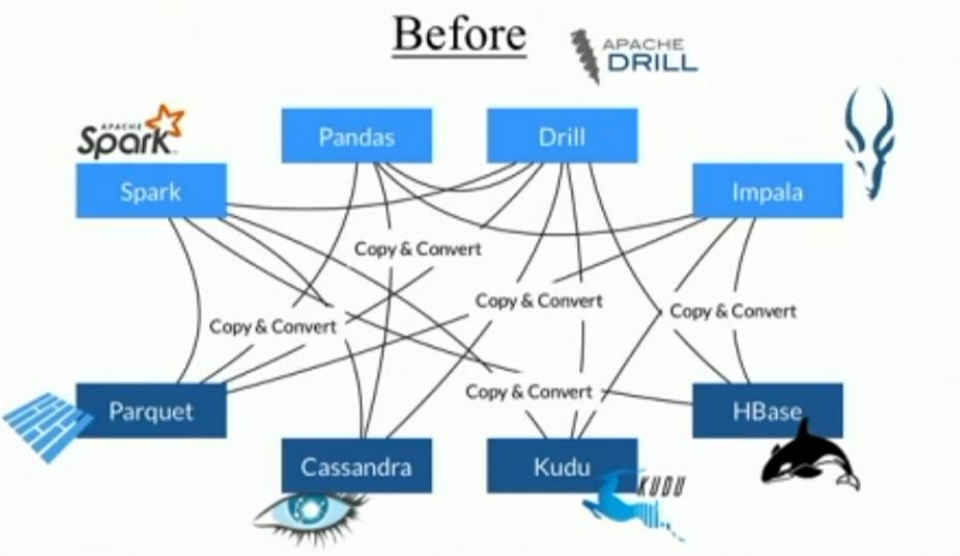
Apportioning of DRAM: Elements

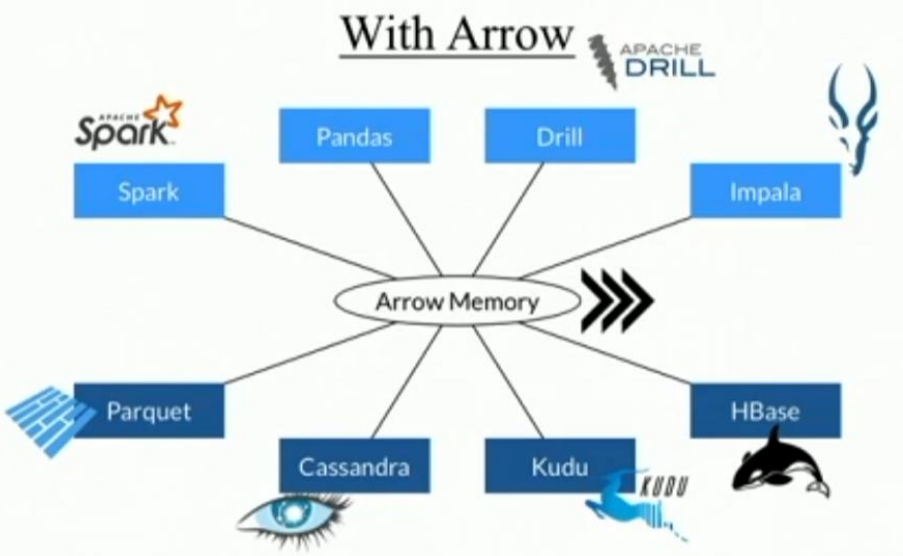
- A process's **Address Space:**
  - Slice of virtualize DRAM assigned to it alone!
  - OS "translates" DRAM vs disk address
- **Page Replacement Policy:**
  - When DRAM fills up, which cached page to evict?
  - Many policies in OS literature
- **Memory Leaks:**
  - Process forgot to "free" pages used a while ago
  - Wastes DRAM and slows down system
- **Garbage Collection:**
  - Some PL implementations can auto-reclaim some wasted memory

Ad: Take CSE 120 or 132C for more on memory management

Storing Data In Memory

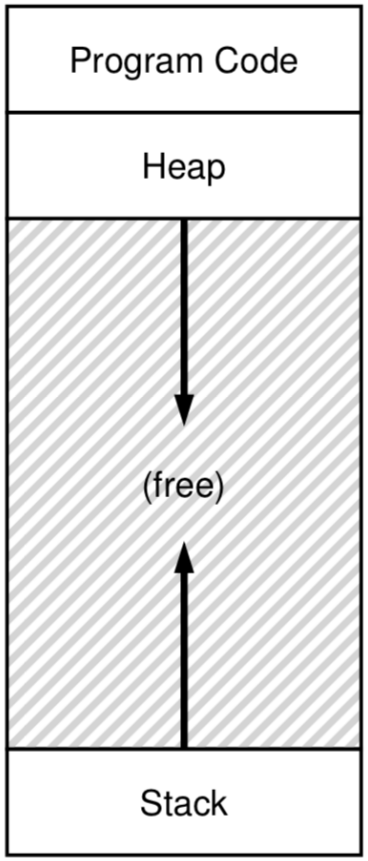
- Any data structure in memory is overlaid on pages
- Process can ask OS for more memory in System Call API
  - If OS denies, process may crash
- **Apache Arrow:**
  - One standard for columnar in-memory data layout
  - Compatible with Pandas, (Py)Spark, Parquet, etc.





Address Space

- Chunk(s) of memory assigned by OS to a process
  - Helps virtualizes and apportion physical memory
- Split into 3 **segments**: Code, Stack, and Heap
  - Stack stores mostly statically known data (function arguments, return values, etc.)
  - Heap is for dynamically created data structures (**malloc()** system call)
  - Stack/Heap can grow/shrink on the fly when a process is running
  - **Segmentation fault**: illegal address access
  - **Memory leak**: program failed to **free()** dynamic space



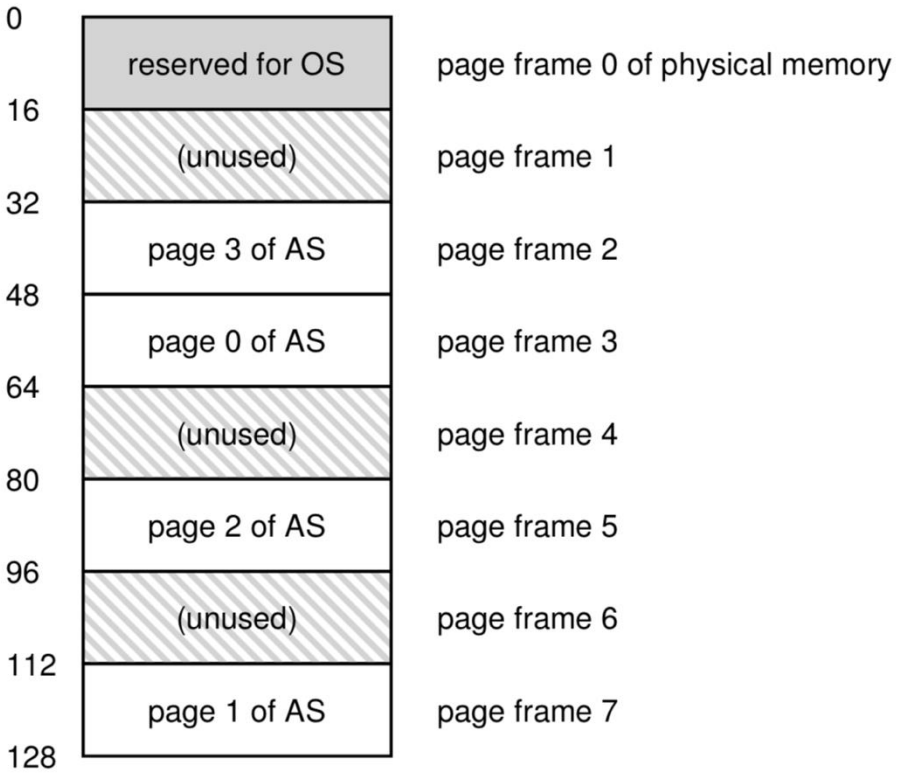
Virtual Memory

- **Virtual** Address vs **Physical** Address:
  - Physical is tricky and not flexible for programs
  - Virtual gives “isolation” illusion when using DRAM
  - OS and hardware work together to quickly perform **address translation**
- OS maintains **free space list** to tell which chunks of DRAM are available for new processes, avoid conflicts, etc.
  - Variable-sized
  - **Fragmentation** possible; algorithms exist to tackle it
- If DRAM space not enough, OS can map virtual address to disk (lower level in memory hierarchy)

Abstraction of Page in Memory

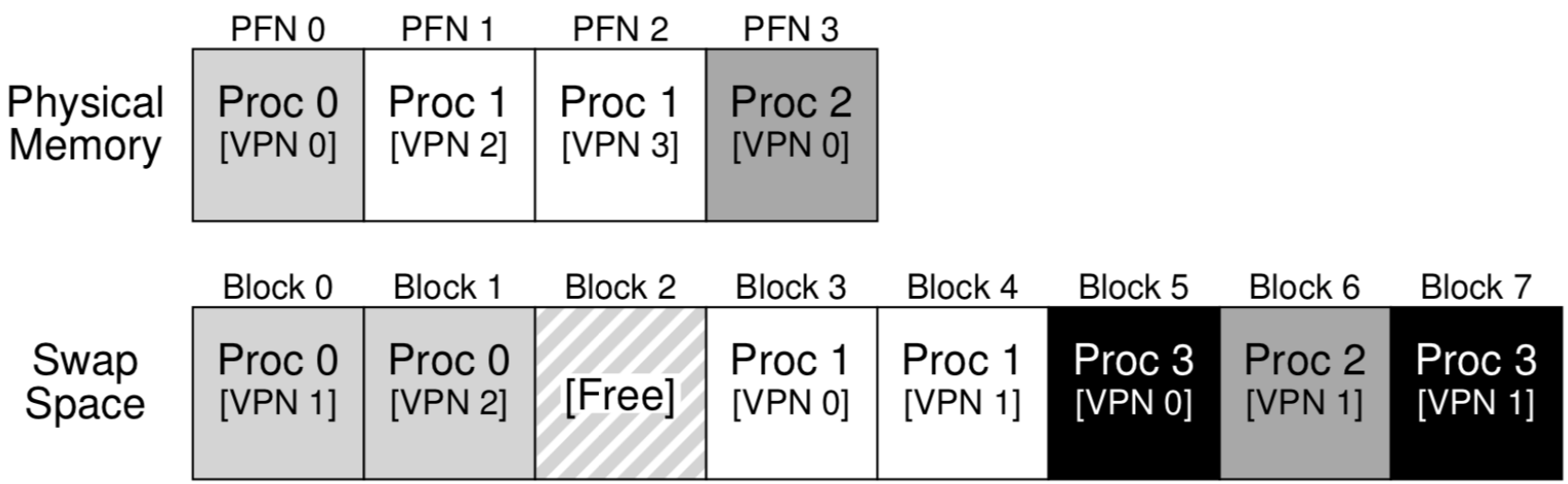
- **Page**: An abstraction of *fixed* size chunks of storage
  - Makes it easier to manage memory virtualization
- **Page Frame**: A virtual "slot" in DRAM to hold a page

- Frame numbers; virtual vs physical page numbers
- OS has **page table** data structure per process to map virtual to physical
- Overall, DRAM chopped up by OS neatly into frames



Swap Space and Paging

- Sometimes, DRAM may not be enough for process(es)
  - OS expands virtual memory idea to disk-resident data
- **Swap Space**: OS reserved space on disk to *swap* pages in and out of DRAM (physical memory)
  - OS should know **disk address** of pages and translate
  - Later: how data is laid out on disks



Page Replacement

- Recall DRAM has page frames to hold page content; a process's address space may only have so many frames
- **Page Fault**: A page required by process is not in DRAM
  - OS intervenes to read page from disk to DRAM
  - If free page frame available in DRAM, all good
- **Page Replacement**: If no frame is free when page fault happens, OS must evict some occupied frame's page!
  - **Page Replacement Policy** (aka cache repl. policy): Algorithm that OS uses to tell what page to evict
  - Various policies exist with different *performance* and *complexity* tadeoffs: FIFO, MRU, LRU, etc. (later topic)

Topic 1 – Outline

- Basics of Computer Organization
  - Digital Representation of Data
  - Processors and Memory Hierarchy
- Basics of Operating Systems (OS)
  - Process Management: Virtualization; Concurrency
  - Main Memory Management
  - Filesystem and Data Files
- Persistent Data Storage

Q: *What is a file?*

Abstractions: File and Directory

- **File:** A persistent sequence of bytes that stores a logically coherent digital object for an application
  - **File Format:** An application-specific standard that dictates how to interpret and process a file's bytes
    - Numerous file formats exist (e.g., TXT, DOC, GIF, MPEG); varying data models/types, domain-specific, etc.
  - **Metadata:** Summary or organizing info about file content (aka *payload*) stored with file itself; format-dependent
- **Directory:** A cataloging structure with a list of references to files and/or (recursively) other directories
  - Typically treated as a *special kind of file.*
  - Common terms: Subdirectory, Parent directory, Root directory.

Filesystem

- **Filesystem:** The part of OS that helps programs create, manage, and delete files on disk (secondary storage)
- Roughly split into *logical level* and *physical level*
  - Logical level exposes file and directory abstractions and offers System Call APIs for file handling
  - Physical level works with disk firmware and moves bytes to/from disk to DRAM

Filesystem

- Dozens of filesystems exist, e.g., ext3, btrfs, NTFS, APFS, etc.
  - Differ on
    - how they layer file and directory abstractions as bytes, what metadata is stored, etc.
    - how data integrity/reliability is assured, support for editing/resizing, compression/encryption, etc.
  - Some can work with (can be "**mounted**" by) multiple OSs.

Why A Filesystem

Turn a flat array of 512-byte sectors into something humans can use.

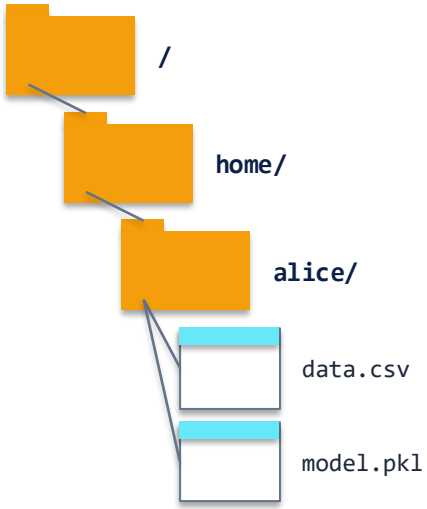
What the disk gives us

512-byte sectors, addressed 0 … n-1

Problems for users:

- No names — just integers.
- No grouping — no notion of “my dataset”.
- No protection, no sharing.
- No metadata: size, owner, modified time…

What the file system delivers



Named bytes. Grouped. Protected. Shared.

Virtualization of File on Disk

- OS abstracts a file on disk as a virtual object for processes
- **File Descriptor:** An OS-assigned positive integer identifier/reference for a file's virtual object that a process can use
  - 0/1/2 reserved for STDIN/STDOUT/STDERR
  - **File Handle:** A programming language's abstraction on top of a file descriptor (fd)

The File Abstraction

File: A linear array of bytes with a name (two names actually) and metadata.

Content: a stream of bytes

Two names for every file

Human name: `/home/alice/data.csv`

Low-level name: `inode number (e.g. 1174)`

The file system translates human names into inode numbers, and inode numbers into bytes on disk.

Metadata (what `stat` shows)

`type`	regular file / directory / symlink
`size`	bytes in the file
`owner / group`	which user, which group
`permissions`	who can read / write / execute
`times`	created / modified / accessed
`link count`	how many names point here
`block map`	pointers to the data blocks on disk

The Directory Abstraction

A directory is just a file whose bytes are (name → inode) pairs.

/home/alice/ (directory contents)

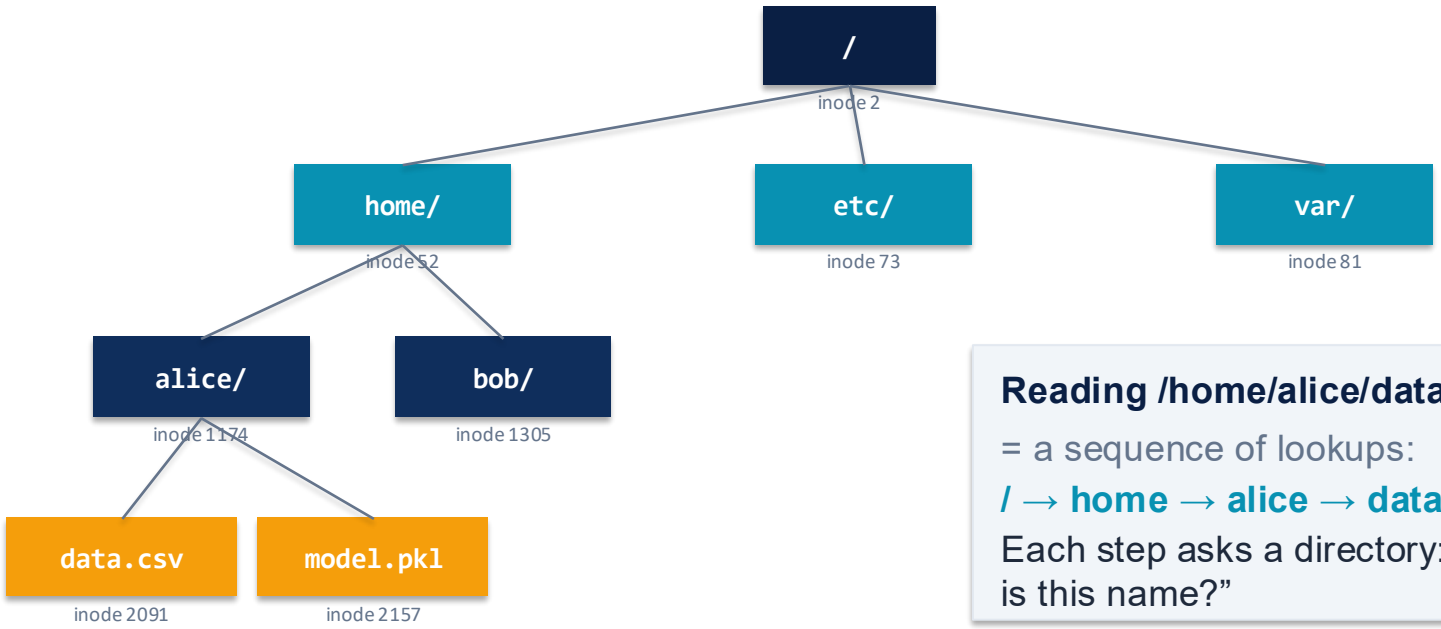
name	inum	what it points to
.	1174	this directory itself
..	52	parent: /home
data.csv	2091	regular file
model.pkl	2157	regular file
notes/	3302	sub-directory
tmp.log	2233	regular file

Key ideas

- **Directories are files.**
  The file system stores a directory's contents in ordinary data blocks.
- **Each entry = (name, inode #).**
  The inode is the true identity; the name is just a label.
- **Every dir has "." and "..".**
  "." points to itself; ".." points to the parent — the backbone of the tree.
- **Names live in dirs, not inodes.**
  An inode does not know its own name. That's why one file can have many names.

The Path Hierarchy

Absolute paths start at "/" and traverse the tree one component at a time.



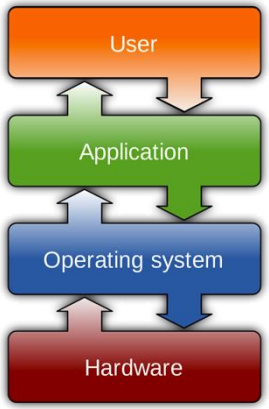
Reading /home/alice/data.csv

= a sequence of lookups:

/ → home → alice → data.csv

Each step asks a directory: “What inode is this name?”

System Call API for File Handling:



API of OS called “System Calls”

- **open**(): Create a file; assign fd; optionally overwrite
- **read**(): Copy file’s bytes on disk to in-mem. buffer; sized
- **write**(): Copy bytes from in-mem. buffer to file on disk
- **fsync**(): “Flush” (force write) “dirty” data to disk
- **close**(): Free up the fd and other OS state info on it
- **lseek**(): Position offset in file’s fd (for random R/W later)
- Dozens more (rename, mkdir, chmod, etc.)

Q: *What is a database? How is it different from just a bunch of files?*

Files vs. Databases: Data Model

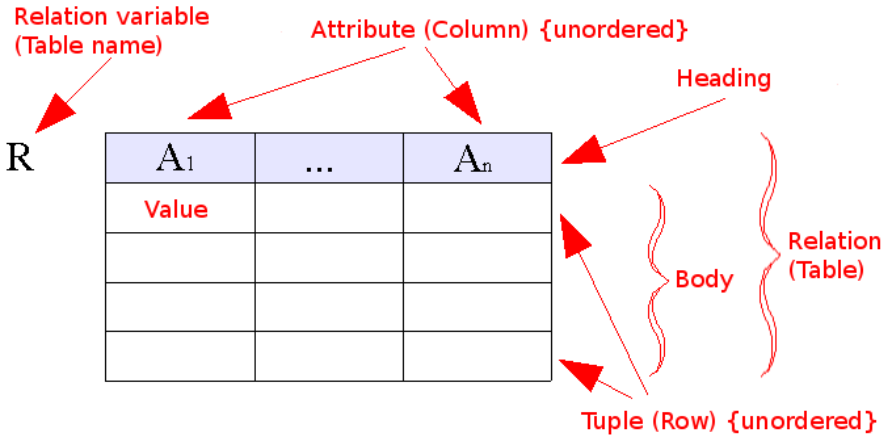
- **Database:** An *organized* collection of interrelated data
  - **Data Model:** An abstract model to define organization of data in a formal (mathematically precise) way

    For example: Relations, XML, Matrices, DataFrames
- Every database is just an *abstraction* on top of data files!
  - **Logical level:** Data model for higher-level reasoning
  - **Physical level:** How bytes are layered on top of files
  - All data systems (RDBMSs, Dask, Spark, TensorFlow, etc.) are application/platform software that use OS System Call API for handling data files

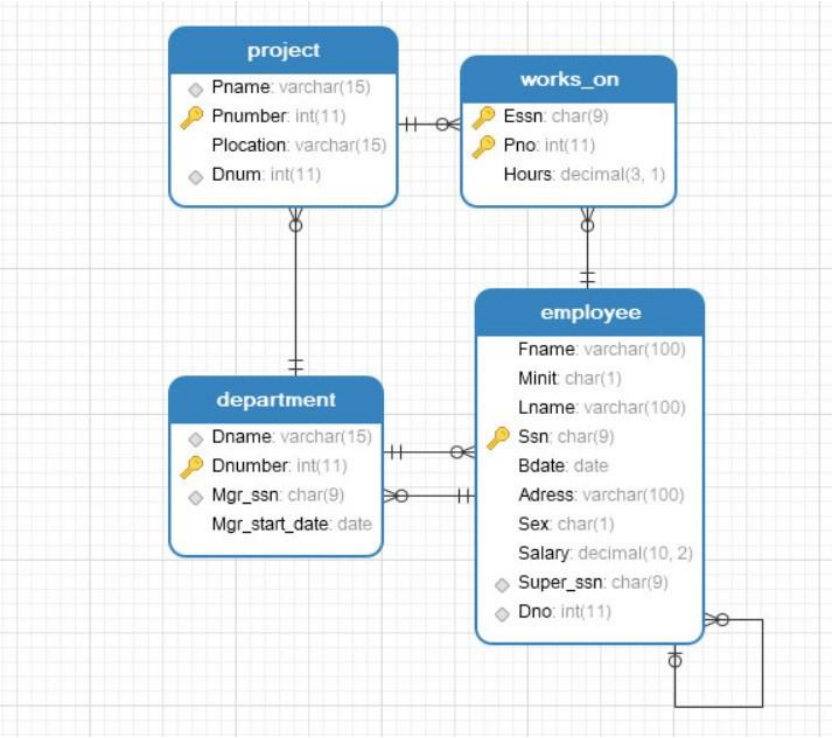
Data as File: Structured

- **Structured Data:** A form of data with regular substructure

Relation



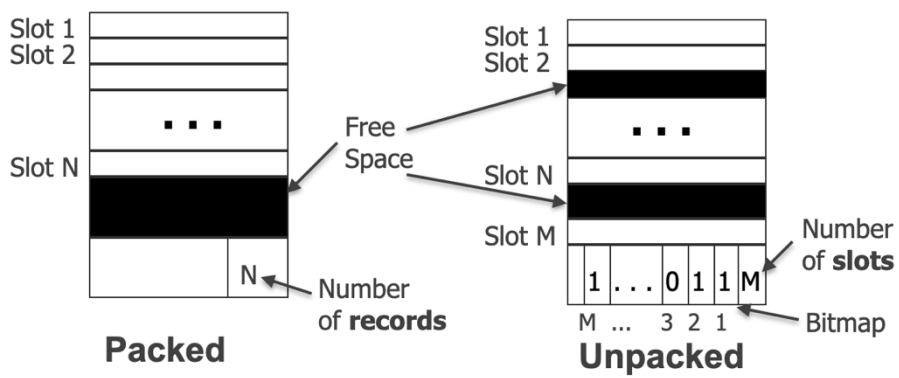
Relational Database



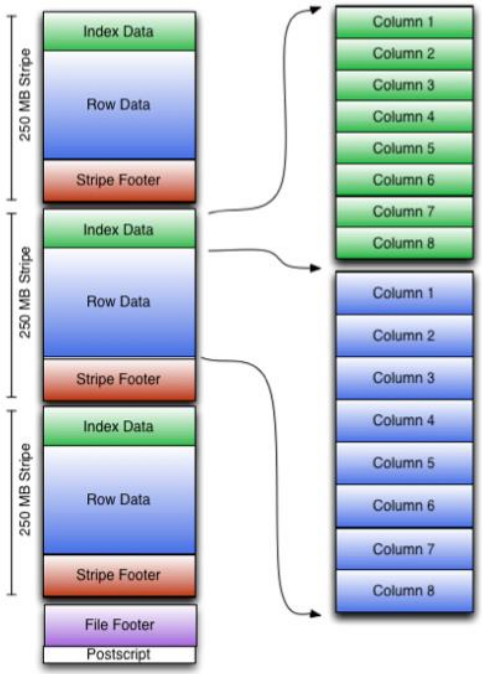
- Relations are serialized by most RDBMSs and Spark as *binary* file(s); often compressed.

Aside: Relational File Formats

- Different RDBMSs and Spark/HDFS-based tools serialize relation/tabular data in different binary formats, often compressed
  - One file per relation; row vs columnar (e.g., ORC, Parquet) vs hybrid formats
  - RDBMS vendor-specific vs open Apache
  - Parquet very popular (used in data lakes)



Row-oriented



Columnar

Ad: Take CSE 132C for more on relational file formats

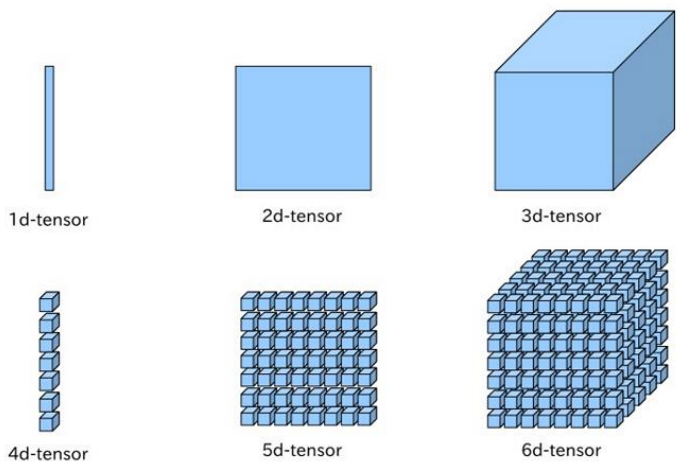
Data as File: Structured

- **Structured Data:** A form of data with regular substructure

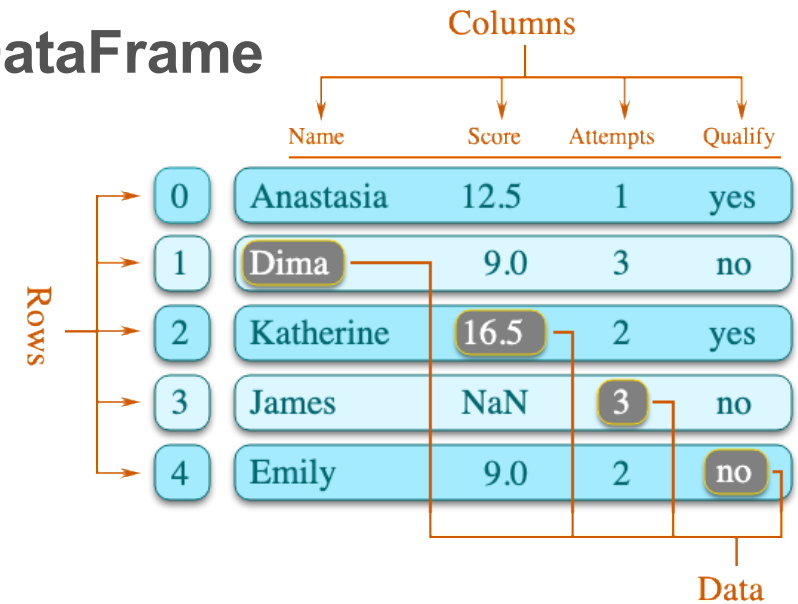
Matrix

$$\begin{array}{c|cccc} & 1 & 2 & \dots & n \\ \hline 1 & a_{11} & a_{12} & \dots & a_{1n} \\ 2 & a_{21} & a_{22} & \dots & a_{2n} \\ 3 & a_{31} & a_{32} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ m & a_{m1} & a_{m2} & \dots & a_{mn} \end{array}$$

Tensor



DataFrame



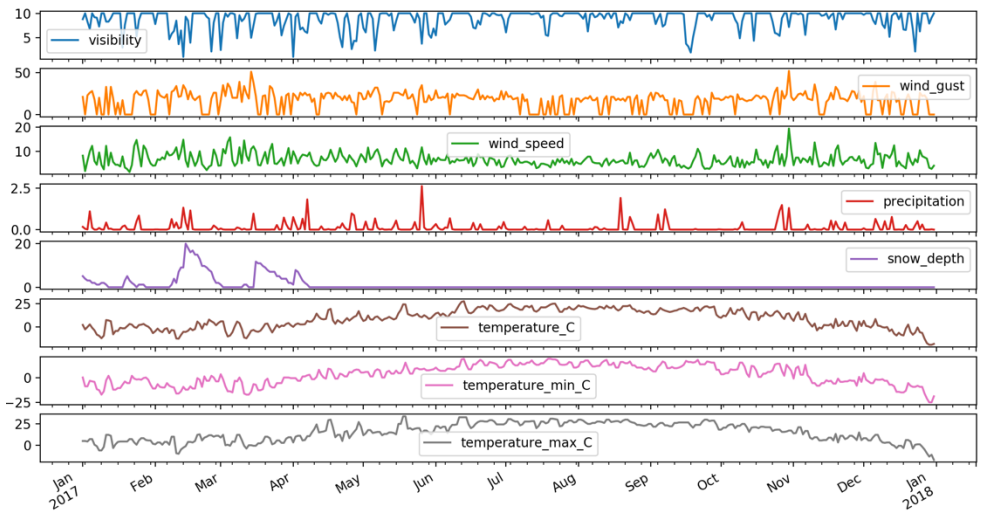
- Typically serialized as restricted ASCII text file (TSV, CSV, etc.)
- Matrix/tensor as binary too
- Can layer on Relations too!

Data as File: Structured

- **Structured Data:** A form of data with regular substructure

Sequence (Includes Time-series)

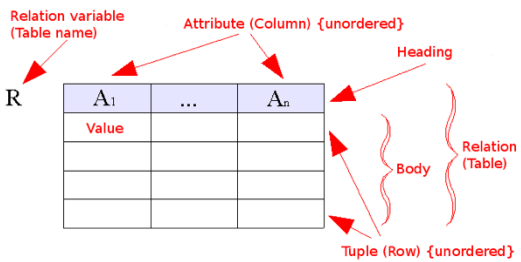
GAT AAAT CT GGTCTTATTTCC



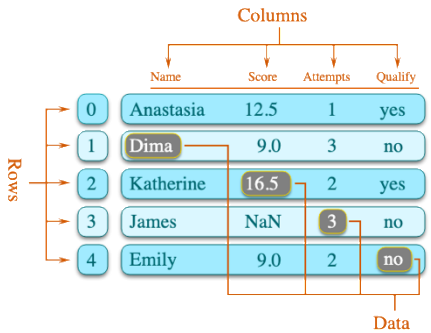
- Can layer on Relations, Matrices, or DataFrames, or be treated as first-class data model
- Inherits flexibility in file formats (text, binary, etc.)

Comparing Structured Data Models

***Q:** What is the difference between Relation, Matrix, and DataFrame?*



$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ \vdots \\ m \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & \cdots & n \end{array} \\ \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ a_{31} & a_{32} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \end{array}$$



- **Ordering:** Matrix and DataFrame have row/col numbers; Relation is orderless on both axes!
- **Schema Flexibility:** Matrix cells are numbers. Relation tuples conform to pre-defined schema. DataFrame has no pre-defined schema but all rows/cols can have names; col cells can be mixed types!
- **Transpose:** Supported by Matrix & DataFrame, not Relation

If interested in reading more:
https://towardsdatascience.com/preventing-the-death-of-the-dataframe-8bca1c0f83c8

Data as File: Semistructured

- **Semistructured Data:** A form of data with less regular / more flexible substructure than structured data

Tree-Structured

```xml
<?xml version="1.0" encoding="UTF-8"?>
<customers>
    <customer>
        <customer_id>1</customer_id>
        <first_name>John</first_name>
        <last_name>Doe</last_name>
        <email>john.doe@example.com</email>
    </customer>
    <customer>
        <customer_id>2</customer_id>
        <first_name>Sam</first_name>
        <last_name>Smith</last_name>
        <email>sam.smith@example.com</email>
    </customer>
    <customer>
        <customer_id>3</customer_id>
        <first_name>Jane</first_name>
        <last_name>Doe</last_name>
        <email>jane.doe@example.com</email>
    </customer>
</customers>
```

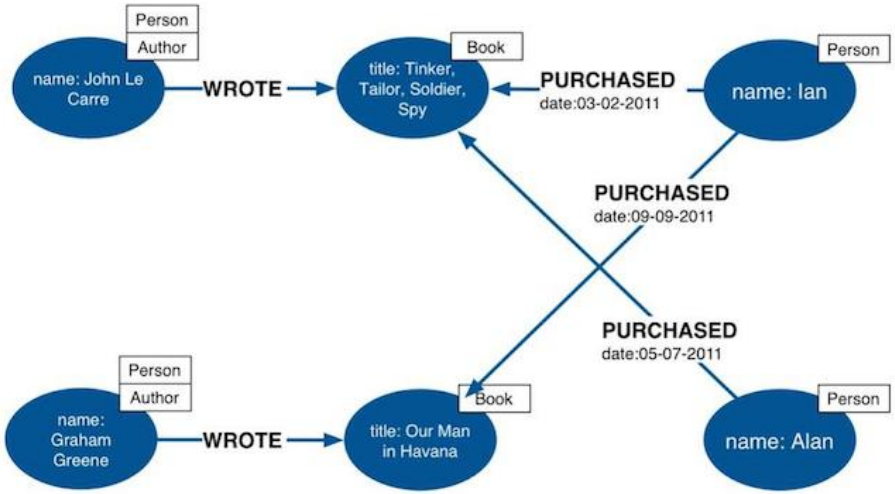
```
[
    {
        orderId: 1,
        date: '1/1/2014',
        orderItems: [
                {itemId: 1, qty: 3, price: 23.4},
                {itemId: 23, qty: 2, price: 3.3},
                {itemId: 7, qty: 5, price: 5.3}
            ]
    },
    {
        orderId: 2,
        date: '1/2/2014',
        orderItems: [
            {itemId: 31, qty: 7, price: 3.8},
            {itemId: 17, qty: 4, price: 9.2}
        ]
    },
    {
        orderId: 3,
        date: '1/5/2014',
        orderItems: [
            {itemId: 11, qty: 9, price: 13.3},
            {itemId: 27, qty: 2, price: 19.2},
            {itemId: 6, qty: 19, price: 3.6},
            {itemId: 7, qty: 22, price: 9.1}
        ]
    }
]
```

- Typically serialized as restricted ASCII text file (extensions XML, JSON, YML, etc.)
- Some data systems also offer binary file formats
- Can layer on Relations too

Data as File: Semistructured

- **Semistructured Data:** A form of data with less regular / more flexible substructure than structured data

Graph-Structured

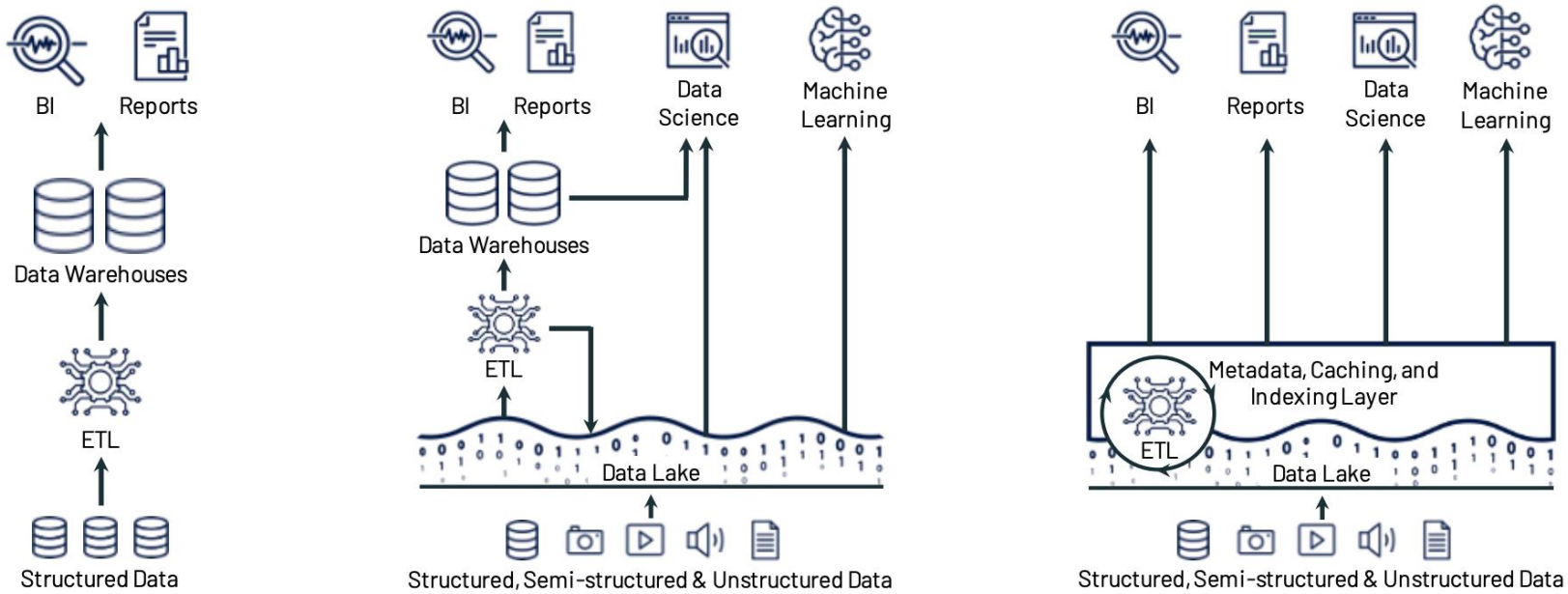


- Typically serialized with JSON or similar textual formats
- Some data systems also offer binary file formats
- Again, can layer on Relations too

Ad: Take DSC 104 for more on semistructured data

Data Files on Data "Lakes"

- **Data "Lake":** *Loose coupling* of data file format for storage and data/query processing stack (vs RDBMS's tight coupling)
  - JSON for raw data; Parquet processed is common



(a) First-generation platforms. **(b) Current two-tier architectures.** **(c) Lakehouse platforms.**

ETL: Extract, Transform, Load

Vision paper on the future of data lakes: http://cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf

Data Lake File Format Tradeoffs

- Pros and cons of Parquet vs text-based files (CSV, JSON, etc.):
  - **Less storage**: Parquet stores in **compressed** form; can be much smaller (even 10x); less I/O to read
  - **Column pruning**: Enables app to read only columns needed to DRAM; even less I/O now!
  - **Schema on file**: Rich metadata, stats inside format itself
  - **Complex types**: Can store them in a column
  - **Human-readability**: Cannot open with text apps directly
  - **Mutability**: Parquet is immutable/read-only; no in-place edits
  - **Decompression/Deserialization overhead**: Depends on application tool; can go either way
  - **Adoption in practice**: CSV/JSON support more pervasive but Parquet is catching up

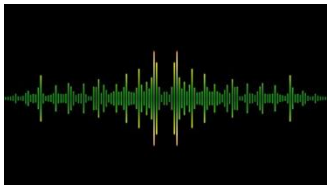
Data Lake File Format Tradeoffs

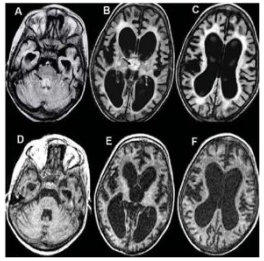
Dataset	Size on Amazon S3	Query Run Time	Data Scanned	Cost
Data stored as CSV files	1 TB	236 seconds	1.15 TB	$5.75
Data stored in Apache Parquet Format	130 GB	6.78 seconds	2.51 GB	$0.01
Savings	87% less when using Parquet	34x faster	99% less data scanned	99.7% savings

https://databricks.com/glossary/what-is-parquet

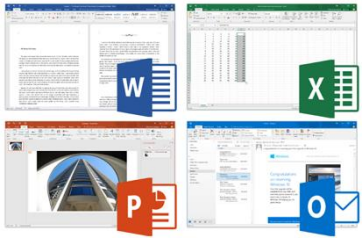
Data as File: Other Common Formats

- **Machine Perception** data layer on tensors and/or time-series
- Myriad binary formats, typically with (lossy) compression, e.g., WAV for audio, MP4 for video, etc.





- **Text File** (aka plaintext): Human-readable ASCII characters
- **Docs/Multimodal File:** Myriad app-specific rich binary formats



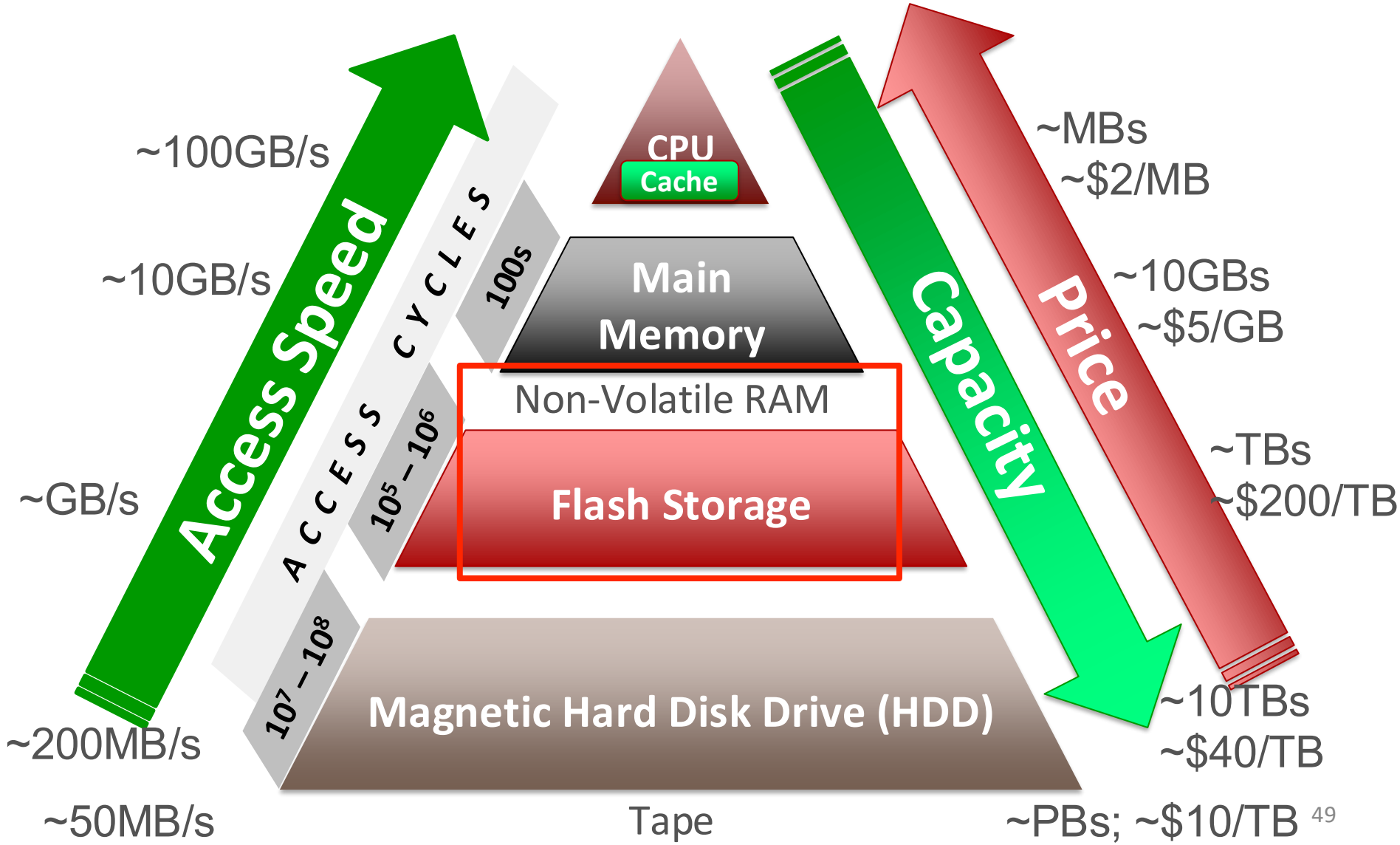
Topic 1 – Outline

- Basics of Computer Organization
  - Digital Representation of Data
  - Processors and Memory Hierarchy
- Basics of Operating Systems (OS)
  - Process Management: Virtualization; Concurrency
  - Filesystem and Data Files
  - Main Memory Management
- Persistent Data Storage

Persistent Data Storage

- **Volatile Memory**: A data storage device that needs power/electricity to store bits; e.g., DRAM, CPU caches (SRAM)
- **Persistence**: Program state/data is available intact even after process finishes
- **Non-Volatile** or **Persistent** memory/storage: A data storage device that retains bits intact after power cycling
  - E.g., all levels below DRAM in memory hierarchy
  - "**Persistent Memory** (PMEM)": Marketing term for large DRAM that is backed up by battery power!
  - **Non-Volatile RAM (NVRAM)**: Popular term for DRAM-like device that is genuinely non-volatile (no battery)

Memory/Storage Hierarchy



Flash SSD vs. Magnetic Hard Disks

Roughly speaking: Flash combines
the speed benefits of DRAM with persistence of disks

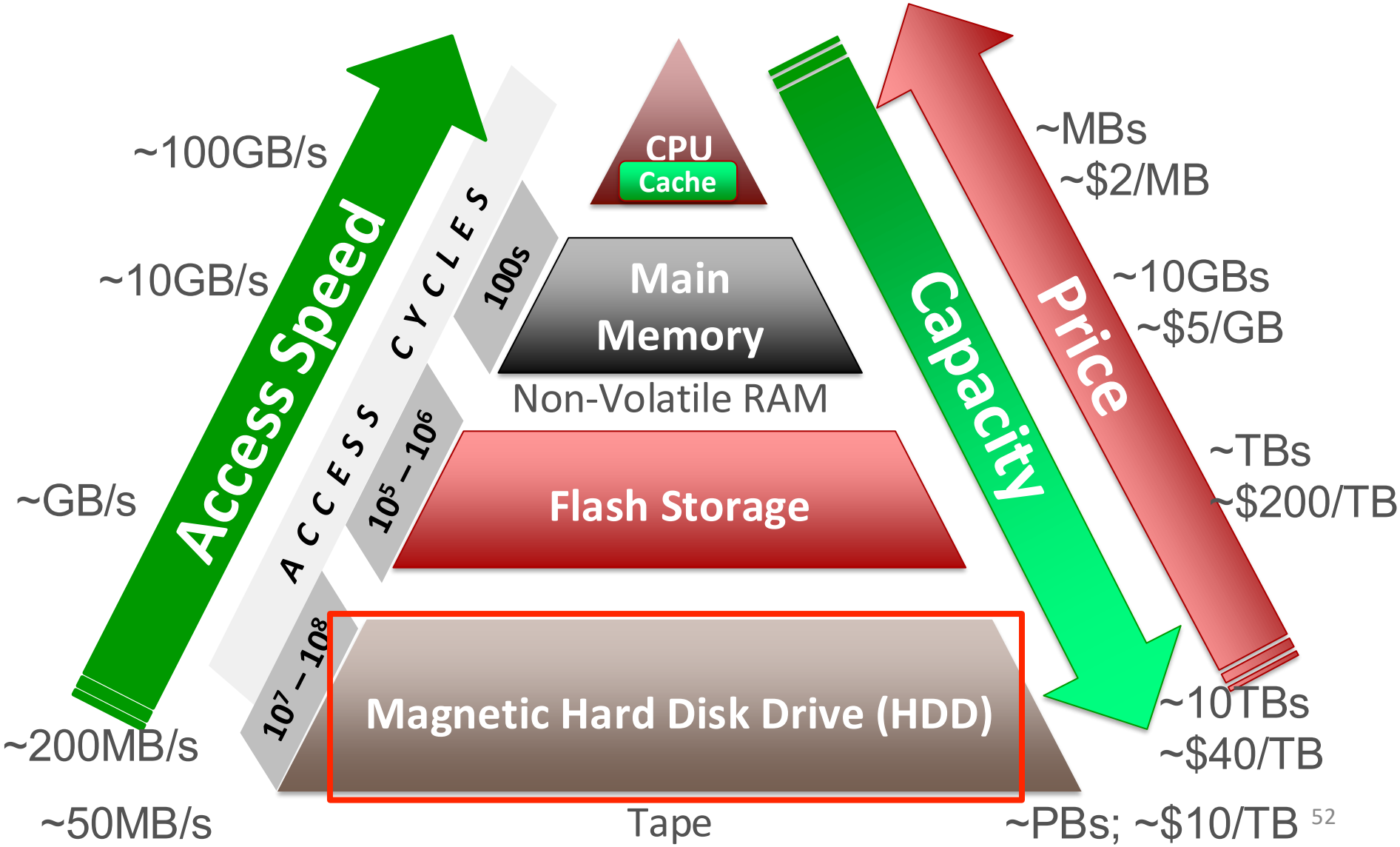
- Random reads/writes are not much worse
  - Different locality of reference for data/file layout
  - But still block-addressable like HDDs
- Data access latency: 100x faster!
- Data transfer throughput: Also 10-100x higher
- Parallel read/writes more feasible
- Read-write impact asymmetry; much lower lifetimes
- Cost per GB has been dropping to 2x-5x higher than HDD (lower than the 5x-15x range of several years ago)
  ...And then AI happened (RAM & SSD prices ballooned in 2025)

NVRAM vs. Magnetic Hard Disks

Roughly speaking: NVRAM is like a non-volatile form of DRAM, but with similar capacity as SSDs

- Random R/W with less to no SSD-style wear and tear
  - Byte-addressability (not blocks like SSDs/HDDs)
  - Spatial locality of reference like DRAM; radical change!
- Latency, throughput, parallelism, etc. similar to DRAM
- Alas, yet to see light of day in production settings
- Cost per GB: No one knows for sure yet!

Memory/Storage Hierarchy



Disks

- Aka secondary storage; likely holds the vast majority of the world's day-to-day business-critical data!
- Data storage/retrieval units: **disk blocks** or **pages**
- Unlike RAM, different disk pages have different retrieval times based on location:
  - Need to *optimize* layout of data on disk pages
  - Orders of magnitude performance gaps possible

Data Organization on Disk

- Disk space is organized into **files**
- Files are made up of disk **pages** aka **blocks**
- Typical disk block/page size: 4KB or 8KB
  - Basic unit of reads/writes for a disk
  - OS/RAM page is *not* the same as disk page!

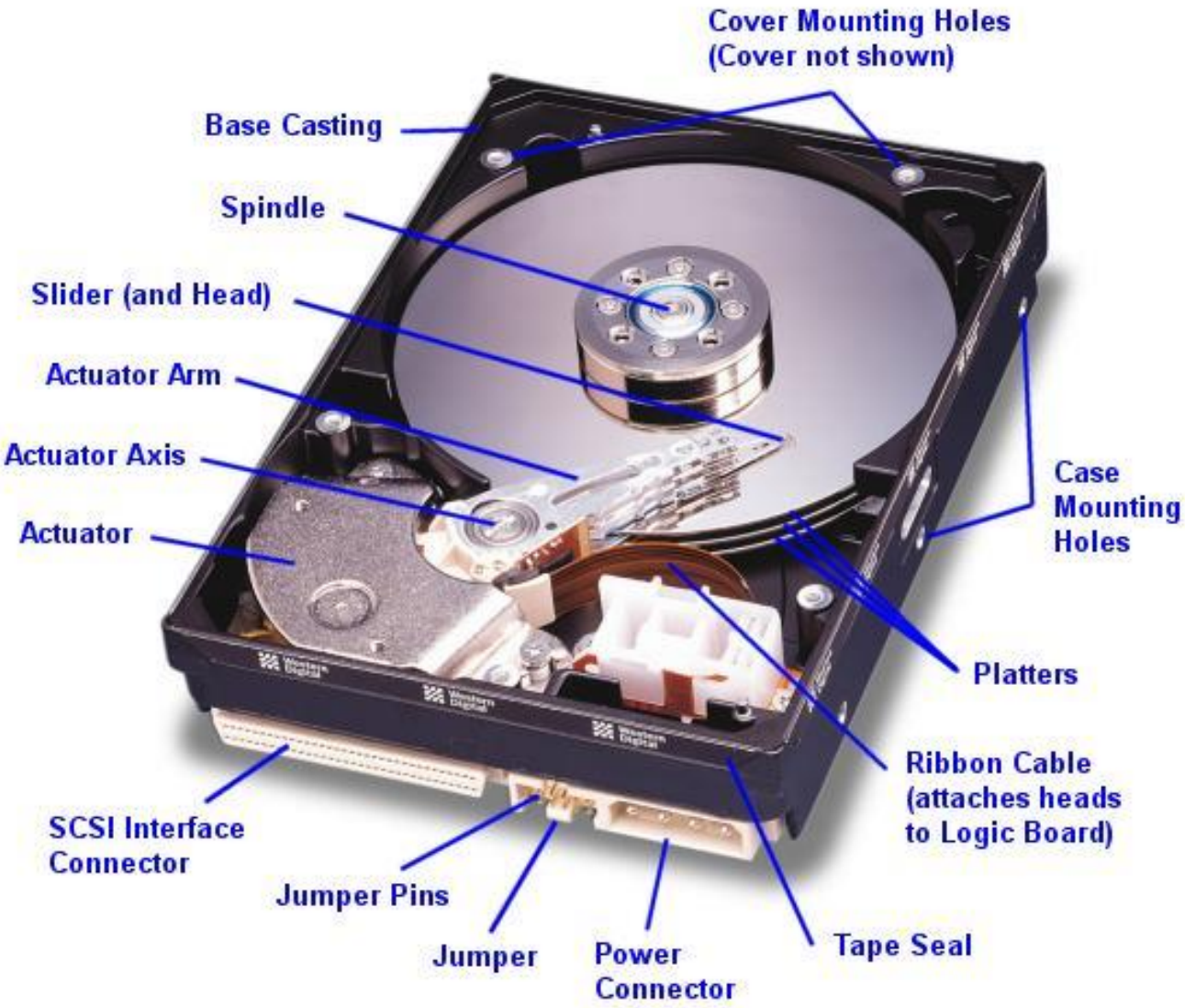
    Typically, [OS/RAM page size] = [Disk page size] but not always; disk page can be a multiple, e.g., 1MB
- File data (de-)allocated in increments of disk pages

Magnetic Disk Quirks

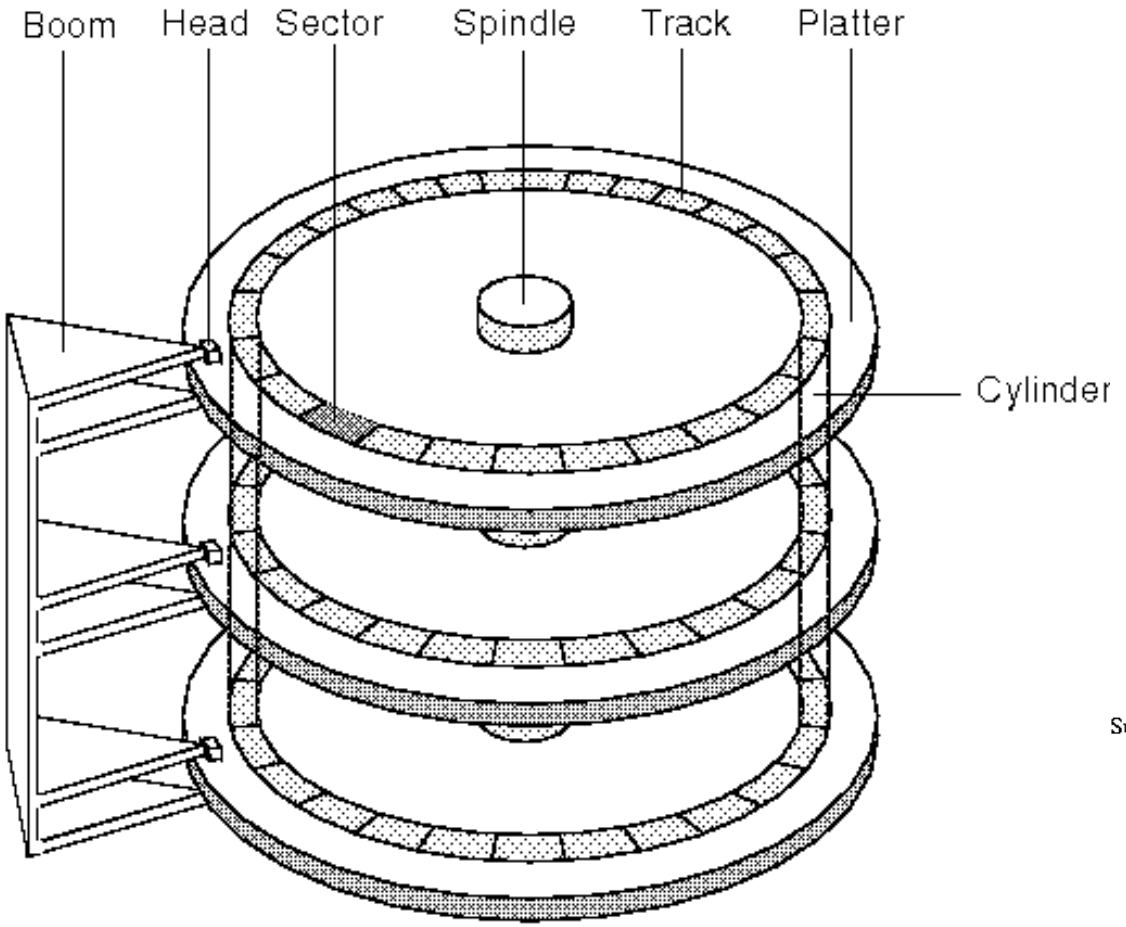
- **Key Principle**: Sequential vs. Random Access Dichotomy
- Accessing disk pages in sequential order gives *higher throughput*
  - Random reads/writes are OOM slower!
- Need to carefully lay out data pages on disk
- Abstracted away by data systems: Dask, Spark, RDBMSs, etc.

Ad: Take CSE 132C for more on quirks of magnetic disks

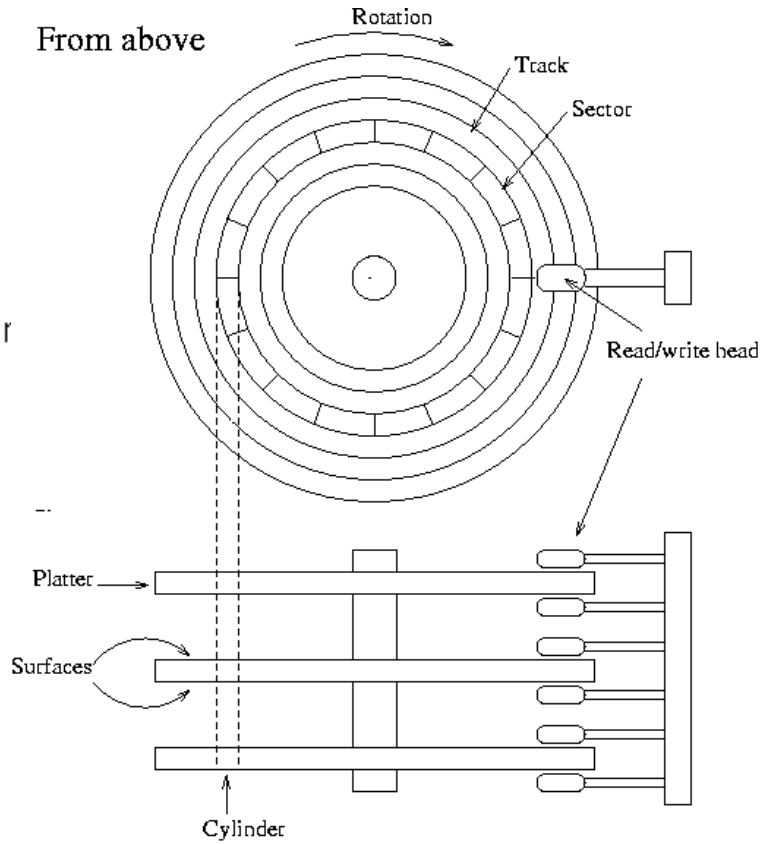
Components of a Disk



Components of a Disk

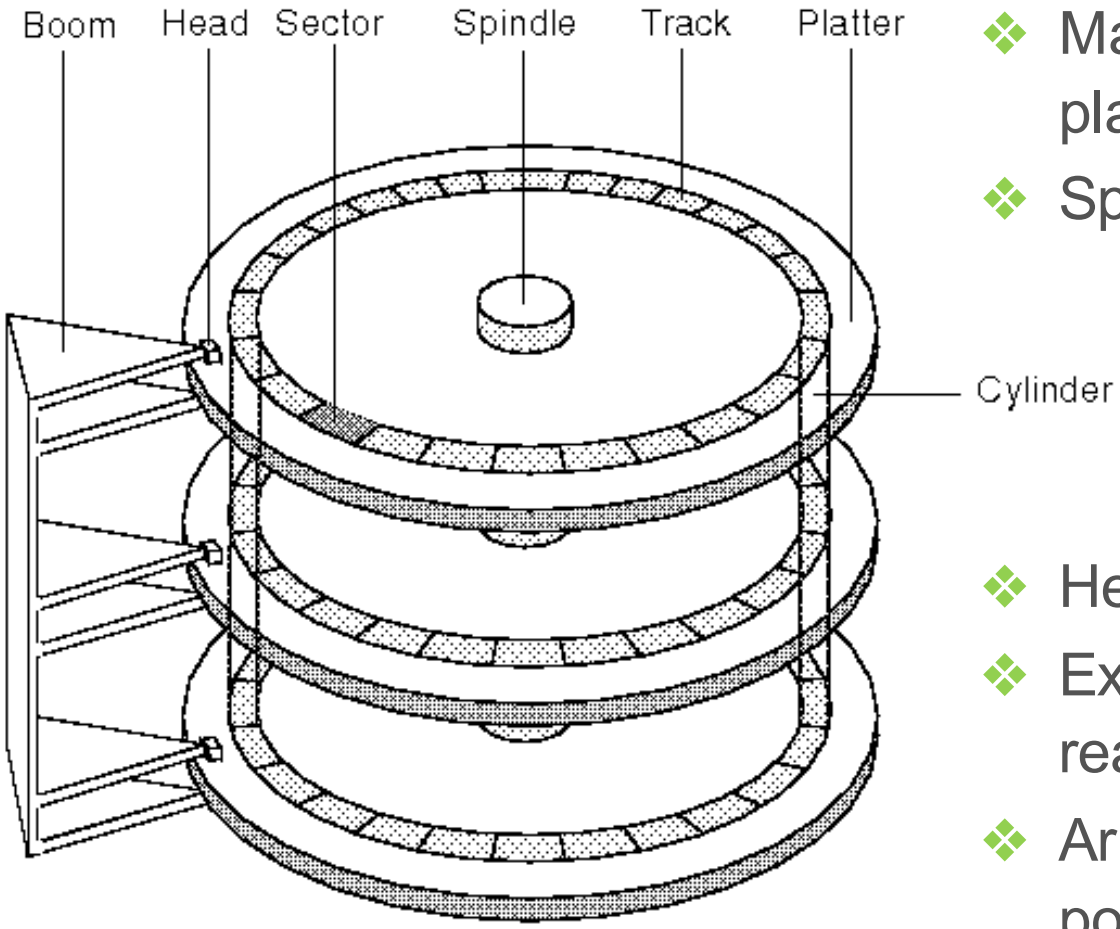


Anatomy of a regular hard disk



1 block = n contiguous sectors
(n fixed during disk configuration)

How does a Disk Work?



Anatomy of a regular hard disk

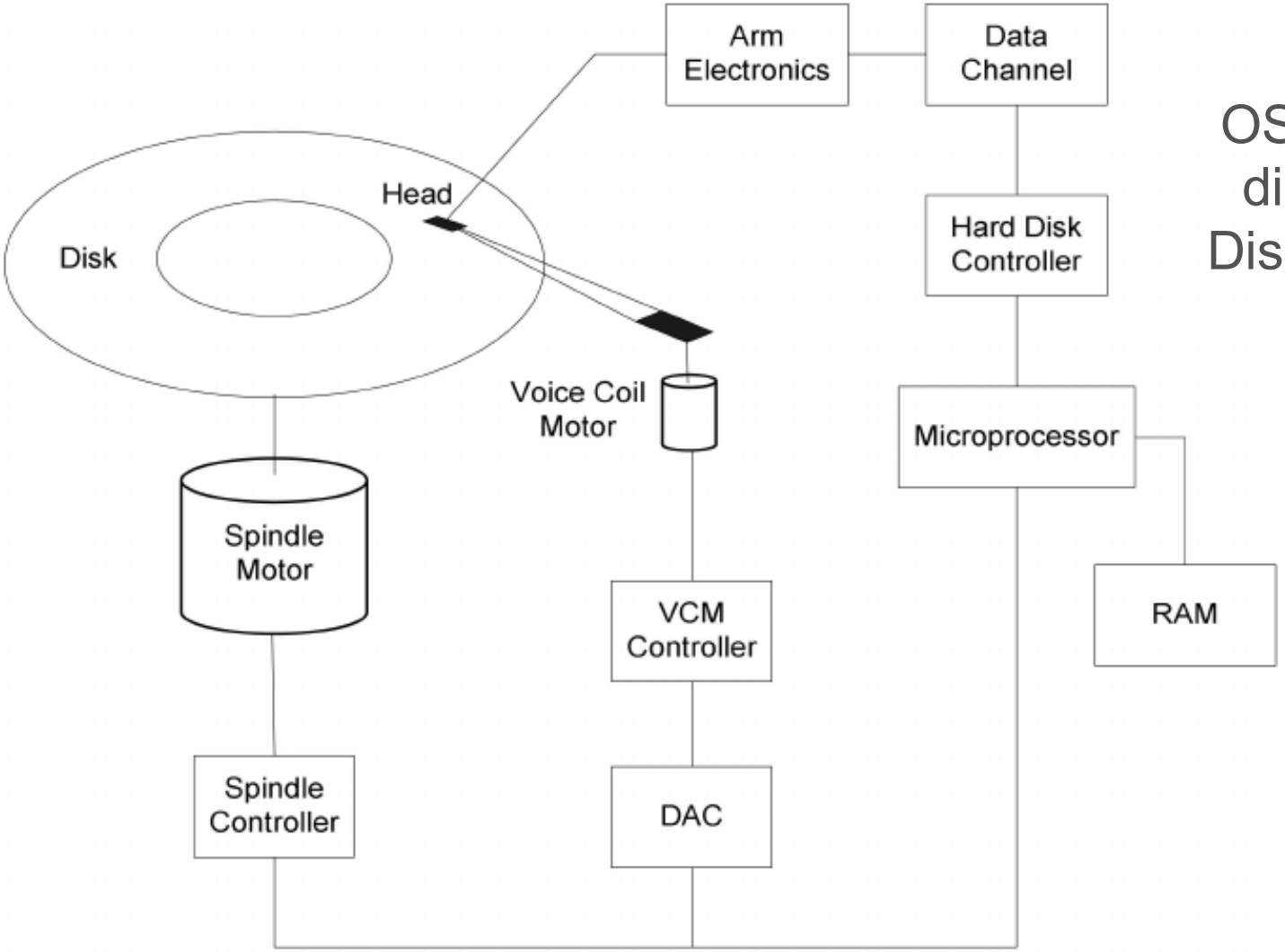
- Magnetic changes on platters to store bits
- Spindle rotates platters

7200 to 15000 RPM

(Rotations Per Minute)

- Head reads/writes track
- Exactly 1 head can read/write at a time
- Arm moves radially to position head on track

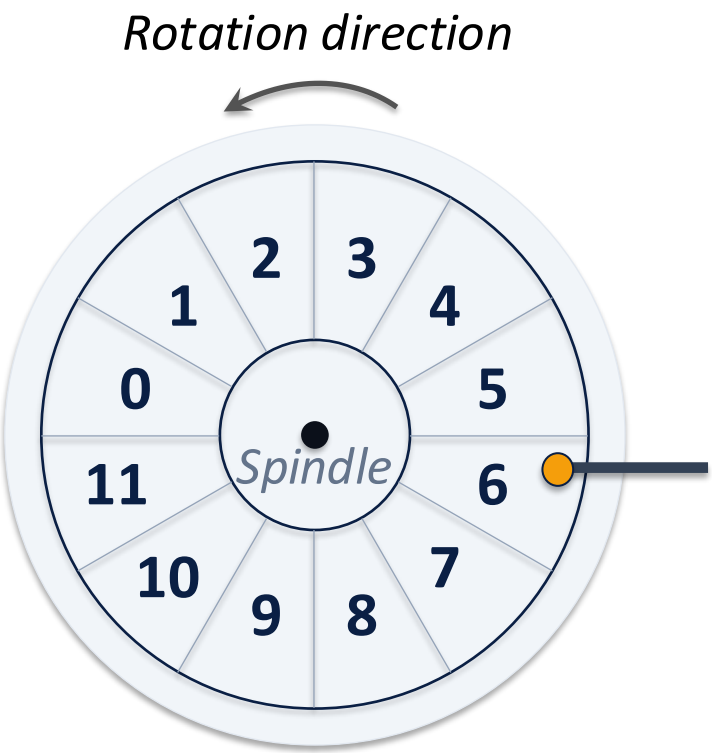
How is the Disk Integrated?



OS interfaces directly with Disk Controller

A Single Track With A Head

12 sectors (0–11) + a read/write head positioned over sector 6.



- **Rotational Delay**

  With only one track, the arm never has to move.

  The head just waits for the target sector to rotate underneath it.

Worst Case

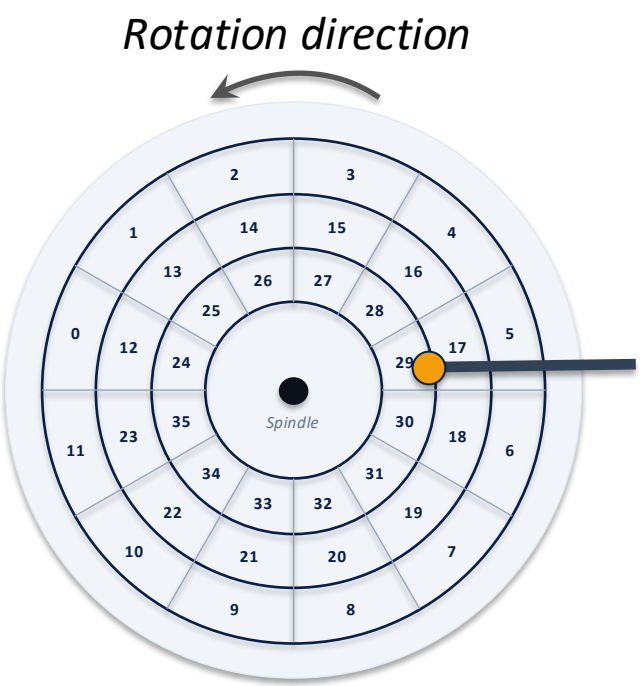
Read block 5 with head over 6 → nearly a full rotation (R).

On average

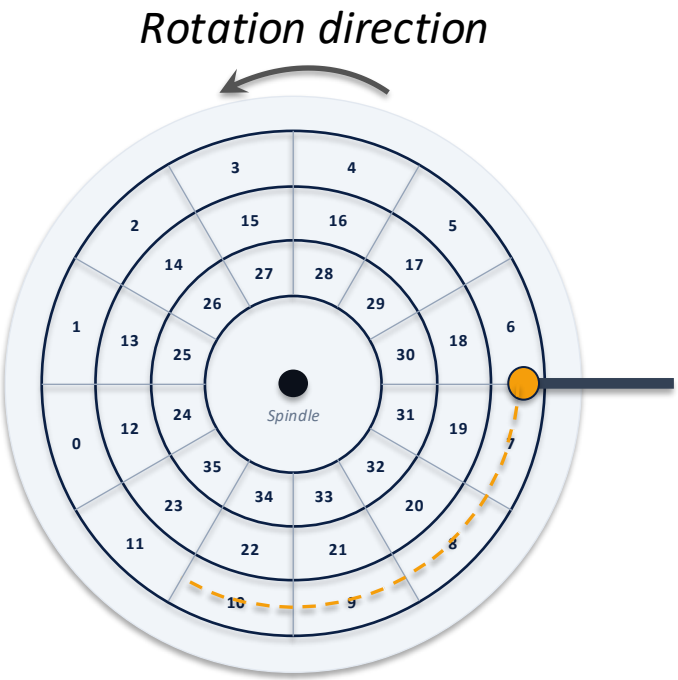
Expect R / 2 of rotation before the sector arrives at the head.

Multiple Tracks And The Seek

The arm must seek to the target track, then wait for rotation.
Seeks are among the costliest disk operations:
accelerate → coast → decelerate → settle.



Before seek: Head on inner track



After seek: Head on outer track, rotating to sector 11

Total I/O = seek + rotation + transfer

The Three Costs Of I/O

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

Seek	Rotation	Transfer
~4 ms	**~2 ms**	**~30 µs**
move the arm	*wait for sector*	*read/write bytes*
Accelerate, coast, decelerate, settle. Average seek ≈ ⅓ of a full seek.	Average = R/2 , where R is one full rotation. 15,000 RPM → R ≈ 4 ms.	Very small for small I/Os; dominates only for long sequential reads or writes.

$R_{I/O}$ = [size of transfer] / $T_{I/O}$

Use dimensional analysis to turn RPM into ms.

Contiguous Vs. Scattered

Why layout matters:
The same 100 MB of data can lead to very different wall-clock time.

SCATTERED (random 4 KB reads)

Each arc = full seek + rotation cost

~6 ms / 4 KB

≈ 0.66 MB/s throughput (Cheetah, random workload)

100 MB → ~20 minutes

CONTIGUOUS (one big sequential read)

One seek + one rotation, then pure transfer

~800 ms / 100 MB

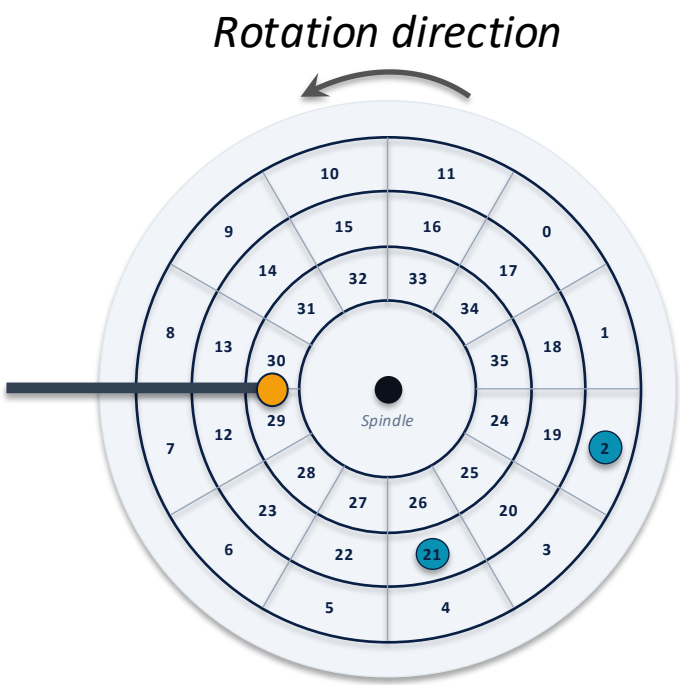
≈ 125 MB/s throughput (Cheetah, sequential)

100 MB → ~0.8 seconds

For data pipelines:
Batch, sort, and stage data so the disk sees long sequential runs, not random hops.

Disk Scheduling

If we must move the arm, then move it cleverly. Pick shortest job first.



SSTF schedules request 21, then request 2

SSTF — Shortest Seek Time First

Pick the pending request whose track is closest to the head.

Watch out: Can starve far-away tracks when near-tracks keep arriving.

SCAN / C-SCAN (the "elevator")

Sweep across tracks in order (out→in→out, or only one direction).

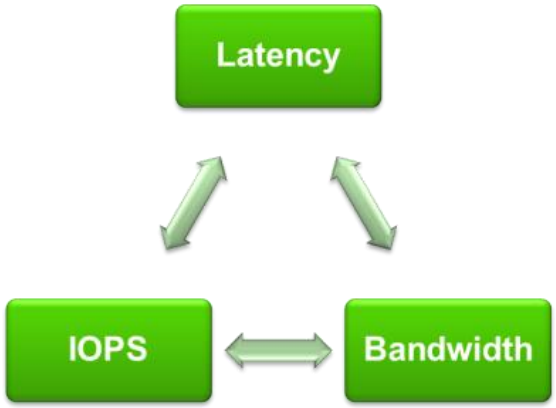
Watch out: Prevents starvation, but ignores rotation — not truly SJF.

SPTF — Shortest Positioning Time First

Account for seek AND rotation; pick request with least total time.

Watch out: Hard in the OS — done inside the drive today. OS also does I/O merging.

Measures Of Storage Performance



- **IOPS**

IO operations per second that the storage subsystem can sustain. Specified differently for reads vs writes.

- **Bandwidth/Throughput**

Bytes/sec that one can achieve across all requests to the storage subsystems.
[Bytes/sec] = IOPS x [Size of IO].

- **Latency**

Total time needed to deliver the data to the application. Includes OS delays, storage delays and (possibly)network delays.

- **Communication/Network I/O Cost**

Abstract counting of number of pages/bytes sent/received over network.

Think: Quantifying I/O: Disk & Network

Example: Assume a data file of size 40 GB; page size is 4KB

I/O cost to read file is $40\times10^9 / 4\times10^3 = 10\times10^6$ or 10 million page I/Os

A disk with I/O throughput 800 MBps needs
40 GB / 800 MBps = $40\times10^9 / 8\times10^8 = 50$ sec

A network with speed 1.6 Gbps (= 0.2 GBps = 200 MBps) needs
40 GB / 200 MBps = $40\times10^9 / 2\times10^8 = 200$ sec

Think: Quantifying I/O: Disk & Network

Example: Compute disk transfer time for a 10 GB file on a 7200 RPM drive.

We are given that:
File size = 10 GB = 10240 MB
Drive rotation is 7200 RPM, rotation period is 60 sec / 7200 = 60×10^3 ms / 7.2×10^3 = 8.33 ms
Average rotational latency is 8.33 / 2 = 4.17 ms
Average seek time is 8 ms
Transfer rate is 150 MB/s at chunks of size 512 KB = 0.5 MB.

Step 1: Time to transfer one chunk is 0.5 MB / 150 MB/s = 0.0033 s = 3.33 ms

Step 2: $T_{I/O}$ = [seek] + [rotational latency] + [transfer] = 8 + 4.17 + 3.33 = 15.5 ms per I/O

Step 3: Number of I/Os is 10240 MB / 0.5 MB = 20,480 I/Os

Step 4: Eventually, the total time is 20,480 × 15.5 ms = 317440 ms = 317 sec ≈ 5.3 min

Note: A sequential transfer only (no seeks) would need 10240 MB / 150 MB/s = 68 sec
The seek overhead makes for a 317 / 68 ≈ **4.7x slower** transfer than pure sequential.

Disk Access Times

Access time = Rotational delay + Seek time + Transfer time

- Rotational delay
  - Waiting for sector to come under disk head
  - Function of RPM; typically, 0-10ms (avg v worst)
- Seek time
  - Moving disk head to correct track
  - Typically, 1-20ms (high-end disks: avg is 4ms)
- Transfer time
  - Moving data from/to disk surface
  - Typically, hundreds of MB/s!

Typical Hard Drive Disk Spec

Western Digital Blue series

Internal SATA HDD and SSD comparison (Amazon prices; 10/2025)

	WD10EZEX (HDD)	WDS100T3B0A (SSD)
Capacity	1TB	1TB
RPM	7200	N/A
Transfer	6 Gb/s	
#Platters	1	N/A
Transfer Rates	(Internal) 150 MB/s	Read: 560 MB/s Write: 520 MB/s
Price	$43	$67

What A DS Should Remember

01 **Every I/O pays three bills**
Seek + rotation + transfer. Seek and rotation dominate for small, random I/Os.

02 **Sequential can beat random by 100–300×**
Layout and access pattern matter more than raw drive speed. Batch, stream, and merge.

03 **"Nearby" is almost free**
The unwritten contract: two blocks on the same track cost roughly one rotation — no seek.

04 **Hardware already helps — but only if you let it**
Track skew, multi-zone layout, track buffers, and SPTF scheduling assume you'll read in order.

05 **Design your data like a file system does**
Co-locate what is read together; sort/bucket/index so pipelines do long sequential sweeps.

Review Questions

- What are the 2 levels of a filesystem? Why the dichotomy?
- How is a database different from a file?
- What are the 2 levels of a database? Why the dichotomy?
- Name 3 forms of structured, 2 forms of semistructured, and 2 forms of unstructured data models.
- Describe 2 differences between a relation and a DataFrame.
- Can you store a relation as a DataFrame? Vice versa?
- Can you store a tensor as a relation? Vice versa?
- What is the address space of a process? What is a memory leak?
- What is Parquet? Explain 3 pros of Parquet over CSVs.
- What is Arrow? How is it different from Parquet? Why are both gaining popularity in practice?

  (See: https://stackoverflow.com/questions/56472727/difference-between-apache-parquet-and-arrow#56481636)
- Name 3 forms of persistent storage devices. Which one has a key latency dichotomy for random vs sequential data access?